

Recitation 05

While Repetition Structure

⇒ { 没有括号, print 函数会自动换行
没有 \n, 因为 print 打印时识别 \n 会自动换行

Question 1 | Predict output

Predict the printed output obtained when running the following programs.

Program 1 <pre> a = "python" b = "" i = len(a) while i > 0: b = b + str(i) + ":" + '{:->{w}}'.format(a[:i], w=len(a)) + "\n" i -= 1 print(b + "0:" + "-" * len(a)) </pre> Handwritten notes: 6: python, 5: -pytho, 4: --pyth, 3: ---pyt, 2: ----py, 1: -----p, 0: -----	Output 1 <pre> 6: python 5: -pytho 4: --pyth 3: ---pyt 2: ----py 1: -----p 0: ----- </pre>
Program 2 <pre> m = 6 n = 3 i = 1 while i < m: j = 1 while j <= i and i <= n: print('*', end=' ') j += 1 j += 1 while j <= m and i > n: print('*', end=' ') j += 1 print() i += 1 </pre> Handwritten notes: i=1, 2, 3, 4, 5; j=1, 2, 3; break; print(end='\n')	Output 2 <pre> * * * * * * * * * * * * * * * </pre>
Program 3 <pre> a = 0 b = 1 while a < 10: print(a) c, a = a, b b = c + b </pre> Handwritten notes: 0 1 1 2 3 5 8, 斐波那契数列	Output 3 <pre> 0 1 1 2 3 5 8 </pre>

a 0 → 1 → 1 → 2 → 3 → 5 → 8 → 13
b 1 → 1 → 2 → 3 → 5 → 8 → 13 → 21 break
c → 0 → 1 → 1 → 2 → 3 → 5 → 8

S 没有重新赋值. 所以是接着 A B C D I 继续的

Question 2 | Display pattern

No definitely right answer / disputable.

Consider the following program:

```
m = 5
n = 3
x = ord('A') = 97
s = ""
i = 1

while i < m:
    j = 1
    while j <= i and i <= n:
        s += chr(x) + '|'
        j += 1
        x += 1
    j += i
    while j <= m and not i > n:
        s += chr(x) + '|'
        j += 1
        x += 1
    print(s)
    i += 1
```

A|B|C|D|

A|B|C|D|

i=2

i=1 ⇒ A|B|C|D|

i=2 ⇒ ~~A|B|C|~~ A|B|C|D

i=3 ⇒ ~~A|B|C|~~

x=97 重新赋值

j=1 → 2 → 3 → 4

j=3 → 4 → 5

5 → 6
100 → 101

What is the printed output? Draw a grid you will use as a guide; each individual character (including blank spaces) of output should be placed in a single box.

```
m = 5
n = 3
x = ord('A')
s = ""
i = 1
while i < m:
    j = 1
    while j <= i and i <= n:
        s += chr(x) + '|'
        j += 1
        x += 1
    j += i
    while j <= m and not i > n:
        s += chr(x) + '|'
        j += 1
        x += 1
    print(s)
    i += 1
```

m=5

n=3

x=97 → 98 → 99 → 100 → 101 → 102 → 103 → 104 → 105 → 106 → 107

i=1 → 2 → 3 → 4 → 5

~~j=1 → 2 → 3 → 4 → 5 → 6~~

~~j=1 → 2 → 3 → 5 → 6~~

~~j=1 → 2 → 3 → 4 → 7~~

~~j=1 → 5 → j=1~~

S = A|B|C|D|

A|B|C|D|E|F|G|

A|B|C|D|E|F|G|H|I|J|

A|B|C|D|E|F|G|H|I|J|

⇒ 就是这样的输出

A	B	C	D							
A	B	C	D	E	F	G				
A	B	C	D	E	F	G	H	I	J	
A	B	C	D	E	F	G	H	I	J	

* string indexing, 用字符串切片功能

Question 3 | Beautiful numbers

用解法转换为列表

- 1) The following numbers are examples of narcissistic number

$$371 = 3^3 + 7^3 + 1^3$$

$$9474 = 9^4 + 4^4 + 7^4 + 4^4$$

A narcissistic number is an n -digit number equal to the sum of its digits raised to the n^{th} power.

Write a program that prompts the user for a positive integer number and check if this number is a narcissistic number.

```
num = input("Enter a number: ")
l = len(num)
i = 0
total = 0
while i < l:
    sep = int(num[i])**l
    total += sep
    i += 1
if total == int(num):
    print(f"{num} is an Armstrong number")
else:
    print(f"{num} is not an Armstrong number")
```

- 2) The following numbers are examples of Kaprekar numbers:

这是 kaprekar number
 $9^2 = 81$, $8 + 1 = 9$;
 $45^2 = 2025$, $20 + 25 = 45$;
 $297^2 = 88209$, $88 + 209 = 297$;
 $4879^2 = 23804641$, $238 + 4641 = 4879$;
 $17344^2 = 300814336$, $3008 + 14336 = 17344$;
 $538461^2 = 289940248521$, $289940 + 248521 = 538461$

To check if a positive whole number n that has d number of digits, check the following:

- Take the square n and separate the result into two pieces: a right-hand piece that has d digits and a left-hand piece that has either d or $d-1$ digits;
- Add these two pieces together.

If the result is n , then n is a Kaprekar number

```
import math
num = int(input("Enter a number: "))
db_number = num**2
left = int(str(db_number)[0:math.floor(len(str(db_number))/2)+1])
+ int(str(db_number)[math.floor(len(str(db_number))/2):len(str(db_number))])
if left == num:
    print(f"{num} is a Kaprekar number")
else:
    print(f"{num} is not a Kaprekar number")
```

Question 4 | String manipulation

- 1) Give in one sentence what the goal of the following program is.

```
N = input('Enter a string: ')
i, j = 0, 0
while i < len(N):
    j = i + 1
    while j < len(N) and N[j] == N[i]:
        j = j + 1
    print(N[i], end='')
    i = j
```

0 0
1 1
2 2
3 3
4 4

Predict what the output will be, given for the input(s) listed in the table. Give your answer

因为没有逗号所以是一样的，所以原样输出

Input	Expected output
abcdeabcdeabcde	abcdeabcdeabcde
aabbccddeeff	abcdeff
abcaabbccaaabbbccdd	abcabcabcd

in the last column of this table.

i 0 → 1 → 2
j 0 → 1 → 2 → 3

→ 这是一个去重的程序 (如果相邻两项一样则去重保留其一个前项)

* string_1 - 开始打代码了

[i:i+3] 必须切片, 不是逗号, 不是元组.

2) Write a program that outputs the number of times a string appear into another.

Example output:

>>>Write a string: abcdeabcdeabcde

>>>Write another string: abc

String 'abc' appears 3 times.

$\text{len('abc')} \Rightarrow$ 只是一个例子

```
string_1 = input("Write a String:")
string_2 = input("Write another String:")
l = len(string_1)
list_1 = list(string_1)
num = 0
for i in range(0, l-2, 1):
    if list_1[i:i+3] == list(string_2):
        num += 1
print(num)
```

$\rightarrow \text{len(string_2)}$

并非永远是3

Question 5 | ISBN checksum

An ISBN (International standard book number) is a number that uniquely identifies commercial books. Before 2007, the ISBNs were 10-digit long and after that year, ISBNs were extended to 13 digits.

In both ISBN-10 and ISBN-13, the last digit of an ISBN is called the *check digit*. The check digit is used to detect erroneous ISBNs:

- In ISBN-10, the check digit can be an integer between 0 and 9 or letter 'X'.
- In ISBN-13, the check digit can be an integer between 0 and 9.

The purpose of this question is to write a program that checks if a number is a valid ISBN-10 or a valid ISBN-13.

For **ISBN-10** numbers, the check digit is computed as follows:

1. First, calculate s , the weighted sum of the first 9 digits with decreasing weights, the weight of the first left digit being 10.
2. Then, calculate n , the number needed to add to s to make it dividable by 11.
3. Finally, if n is equal to 10, the check digit is letter 'X'. Otherwise, the check digit is n .

Example: 0306406152

- weighted sum: $0 \times 10 + 3 \times 9 + 0 \times 8 + 6 \times 7 + 4 \times 6 + 0 \times 5 + 6 \times 4 + 1 \times 3 + 5 \times 2 = 130$
- check digit: $(130 + 2) \% 11 = 0$
- 0306406152 is a valid ISBN-10

For **ISBN-13** numbers, the check digit is computed as follows:

1. First calculate s , the weighted sum of the first 12 digits with alternate weights of 1 and 3, the weight of the first left digit being 1;
2. Then, calculate n , the number needed to add to s to make it dividable by 10.
3. The check digit is n .

Example: 9780306406157

- weighted sum: $9 \times 1 + 7 \times 3 + 8 \times 1 + 0 \times 3 + 3 \times 1 + 0 \times 3 + 6 \times 1 + 4 \times 3 + 0 \times 1 + 6 \times 3 + 1 \times 1 + 5 \times 3 = 93$
- check digit: $(93 + 7) \% 10 = 0$
- 9780306406157 is a valid ISBN-13

Program requirements

- Your program should prompt the user for a number with message "Enter an ISBN number:"
- Your program should check if the number of digits entered are 10 or 13. If not, the output should be "Not a valid ISBN."
- If the number of digits entered is 10 (respectively 13), your program should compute the check digit based on the 9 first digits (respectively the first 12 digits).
- if the check number computed by your program corresponds to the last digit, the output should be "Valid ISBN-10" or "Valid ISBN-13" depending on the number of digits first entered.
- if the check bit computed by your program does not correspond to the last digit entered, the output should be "Not a valid ISBN-10" or "Valid ISBN-13".

Sample output:

```
>>> Enter an ISBN number: 020151141
>>> Not a valid ISBN

>>> Enter an ISBN number: 020151141X
>>> Valid ISBN-10

>>> Enter an ISBN number: 0306406153
>>> Not a valid ISBN-10

>>> Enter an ISBN number: 0306406152
>>> Valid ISBN-10

>>> Enter an ISBN number: 9782340076556
>>> Valid ISBN-13
```

```
dig = input("Enter an ISBN number:")
list = list(dig)
l = len(dig)
weighted_sum = 0

if list[l-1] != 'X':
    weighted_sum = 0
    if l != 10 and l != 13:
        print("Not a valid ISBN")
    elif l == 10:
        for i in range(0, 9):
            weighted_sum += int(list[i]) * (10 - i)
        if (weighted_sum + int(list[9])) % 11 == 0:
            print(f"{dig} is a valid ISBN-10")
        else:
            print(f"{dig} is not a valid ISBN-10")
    elif l == 13:
        for i in range(0, 12):
            if i % 2 == 0:
                weighted_sum += int(list[i]) * 1
            else:
                weighted_sum += int(list[i]) * 3
        if (weighted_sum + int(list[12])) % 10 == 0:
            print(f"{dig} is a valid ISBN-13")
        else:
            print(f"{dig} is not a valid ISBN-13")

if list[l-1] == 'X':
    weighted_sum = 0
    if l != 10:
        print("Not a valid ISBN")
    elif l == 10:
        for i in range(0, 9):
            weighted_sum += int(list[i]) * (10 - i)
        if (weighted_sum + 10) % 11 == 0:
            print(f"{dig} is a valid ISBN-10")
        else:
            print(f"{dig} is not a valid ISBN-10")
```

list 不能当作变量名。
python 内置已有，虽然不影响
但是最好使用 my_list 代替。